

FORMAL VERIFICATION OF NASA-RELATED NEURAL NETWORKS

Sara Logsdon

August 2026

Reading note. This report is written as an engineering case study. It explains the useful results, the attempts that did not work, and the exact limits of every guarantee. All code and reproducibility materials are available at <https://github.com/saralogsdon/neural-network-verification>.

Executive summary

The question behind this project is: if a neural network’s input changes within a clearly stated range, can we prove that its output still behaves as expected? Ordinary testing answers this question for selected examples. Formal verification tries to answer it for every input in the chosen region.

I explored that question with two verification tools, α, β -CROWN and Marabou, and four NASA-related case studies. Both tools reached conclusive, matching results for every final demonstration. The four demonstrations were deliberately chosen to cover different neural-network tasks and different verification boundaries: image classification, regression, text classification, and anomaly detection. In each case, the property was stated as an inequality over a bounded input region and then checked independently with both verifiers. MicroNet preserved the ordering between two outputs while all 512 learned microscopy features changed simultaneously. The Airfoil network kept its lift prediction inside a chosen tolerance band. The NASA text classifier kept Earth Science above every competing label. The GIBS-style anomaly classifier kept a nominal image on the normal side of its binary decision.

The most useful lesson was not that every neural network can already be verified end to end. The full pixel-level MicroNet attempt ran into an unsupported pooling configuration and exceeded the memory available on a Colab T4 GPU. Instead of presenting that as a model failure, I narrowed the claim and stated the new boundary explicitly. This report therefore treats verification as an engineering exercise: choose a meaningful property, identify the exact part of the pipeline being checked, cross-check the encoding, and report only what the evidence actually proves.

Final verified results

Case study	Model provenance	Variables allowed to change	Property proved	Conclusive result
MicroNet	Official pretrained ResNet-18 checkpoint	512 learned features, each within ± 0.083227	Output 20 remains above output 30	α, β -CROWN lower margin 4.713961; Marabou UNSAT
Airfoil	NASA architecture and data; model retrained for the demo	3 normalized operating inputs, each within ± 0.001	Lift stays within ± 0.01 of nominal	α, β -CROWN verified; Marabou lower and upper violation queries both UNSAT
Text	Official NASA-IMPACT RoBERTa classifier checkpoint	768 learned text features, each within ± 0.148291	Earth Science remains above all four competing labels	α, β -CROWN minimum lower margin 2.827524; Marabou 4× UNSAT
GIBS anomaly	NASA GIBS architecture; retrained on real GIBS imagery	256 hue-histogram features, each within ± 0.002594	Normal score remains above anomaly score	α, β -CROWN verified; Marabou UNSAT

1. The question behind formal verification

A test can show that a model behaved correctly on the examples that were run. It cannot, by itself, rule out a failure somewhere else inside a continuous range of possible inputs. Formal verification defines that range mathematically and asks whether a bad output can occur anywhere inside it.

Core pattern. Choose a nominal (unperturbed, baseline) input x_0 . Define an allowed region such as

$$\|x - x_0\|_\infty \leq \varepsilon,$$

and prove an output condition such as

$$f_{\text{pred}}(x) - f_{\text{other}}(x) > 0.$$

In plain language, the equation says: allow every input value to move within a box, then prove that the preferred output stays ahead. The size of the box and the definition of “stays ahead” are choices made by the user.

This also explains why every proof needs a boundary. Showing that output 20 beats output 30 does not show that it beats every other output. Verifying a retrained implementation of a published NASA architecture is not the same as verifying an official released checkpoint. These are not technical footnotes; they determine what the result means.

2. How the two verifiers approach the problem

2.1 α, β -CROWN

α, β -CROWN tries to place a guaranteed lower bound on the safety margin. It propagates linear bounds through the network, tightens them with α -CROWN, and can use β -CROWN branch-and-bound when the first calculation is not strong enough. If the guaranteed lower margin is still above zero, the property is proved. The user can supply a PyTorch or ONNX model and control settings such as the device, timeout, attack order, batch size, and branching behavior [7, 8].

Unlike a SAT- or SMT-style verifier, α, β -CROWN does not primarily translate the network into a logical satisfiability problem and ask a general-purpose solver to find a satisfying assignment. Its core calculation is numerical: it constructs sound linear upper and lower bounds on the network’s activations and on the final specification. For nonlinear units such as ReLUs, CROWN replaces the exact nonlinear relation by linear relaxations that are guaranteed to enclose every possible activation in the current input region. α -CROWN optimizes the slopes of these relaxations to tighten the resulting bounds. If the certified lower bound on the desired margin is already positive, the proof is complete. If not, β -CROWN can split the problem into smaller subdomains and repeat the bounding process in a branch-and-bound search. Thus the certificate comes from sound bound propagation over all inputs in the region, rather than from Boolean satisfiability search [7, 8].

2.2 Marabou

Marabou asks the same question from the opposite direction. Rather than directly proving the safe condition, the code describes what a violation would look like and asks whether such an input exists. If Marabou returns UNSAT, the bad case is impossible inside the allowed region, so the original property is proved. If it returns SAT, it supplies an assignment satisfying the bad condition. Marabou primarily uses ONNX and is designed for feed-forward networks with piecewise-linear operations [9, 10].

The connection to Boolean satisfiability is useful for understanding Marabou’s viewpoint. In the classical SAT problem, one asks whether there is any assignment of **true/false** values that makes a Boolean formula true. SMT, or *satisfiability modulo theories*, extends this idea by allowing variables and constraints from mathematical theories such as real arithmetic. Neural-network verification naturally needs this richer setting because activations are real-valued and the network contains equations, inequalities, and piecewise-linear relations. Marabou encodes the network together with the negation of the desired safety property as a constraint-satisfaction problem. A SAT result means that a concrete violating assignment exists; an UNSAT result means that no such assignment exists in the allowed input region [9, 10].

2.3 Why use both?

Using both tools was valuable because the conclusion appears in two different forms: a positive certified bound from α, β -CROWN, and an UNSAT violation query from Marabou. Agreement provides an independent implementation check. A disagreement would be a reason to inspect the model conversion, input bounds, or inequality direction before trusting the result.

2.4 Options available to the user

User choice	α, β -CROWN	Marabou
Model input	A PyTorch model or an ONNX model	Usually an ONNX model
What inputs can vary?	A range of allowed values for each input	Lower and upper limits for each input variable
What should remain true?	The condition to prove directly, such as a positive margin or an output band	The opposite question: is there any allowed input that breaks the condition?
What can be adjusted?	Device, bound-tightening settings, branching, batch size, and timeout	Timeout, search settings, and the way the violation condition is written
Possible result	Verified, unsafe/counterexample found, or unknown	UNSAT/no counterexample, SAT/-counterexample found, or timeout/unknown
Most useful evidence	A guaranteed worst-case numerical bound or margin	A proof that no violating input exists, or a concrete violating assignment

The most important α, β -CROWN choices are the perturbation bounds, device, timeout, and whether to stop after the initial bound or permit complete branch-and-bound search. Optional projected-gradient attacks can search quickly for a violating input, but a failed attack is not a proof. A user working on a large network may lower the verification batch size, reduce optimization iterations, choose a different branching strategy, or move to a GPU with more memory. These settings change speed and resource use; they do not change the meaning of a sound positive certificate.

For Marabou, the user sets a lower and upper bound for every variable and expresses the unwanted output behavior with equations, inequalities, or disjunctions. A multi-class robustness claim can be encoded as one disjunction or as separate competitor queries. Separate queries are often easier to

explain: every query must return UNSAT to prove that the nominal class beats all competitors. Timeout and verbosity affect the search experience, while SAT and UNSAT retain their formal meanings.

For either tool, model conversion and inequality direction deserve explicit checks. The notebook compares PyTorch and ONNX outputs before verification and evaluates the property at the nominal input. These small checks prevent a technically successful solver run from certifying the wrong model or the reverse of the intended claim.

3. Case study A: NASA MicroNet

3.1 Model and functional purpose

NASA’s MicroNet is a specialized deep learning dataset and collection of pre-trained convolutional neural network models. It contains over 110,000 microscopic images of engineering and structural materials designed to improve image segmentation and quantification in materials science. NASA’s repository includes pretrained image encoders, benchmark segmentation data, and transfer-learning examples.¹ The ResNet-18 v1.0 checkpoint—a ResNet, or *residual network*, is a convolutional architecture built from residual blocks that learn corrections to an identity pathway, allowing substantially deeper image models to train without every layer having to relearn the full representation—used here was trained from scratch on MicroNet; the repository reports 79.815% top-1 accuracy on the original classification task [1].

For the nominal microscopy image used in the final demonstration, the model’s highest output was output 20 and its runner-up was output 30. The verification question was: if the learned description of that image changes slightly, can output 30 catch or overtake output 20? The image-to-feature encoder is the ResNet-18 convolutional backbone; the formally verified component is the exact released linear head from $z \in \mathbb{R}^{512}$ to $f(z) \in \mathbb{R}^{1000}$.

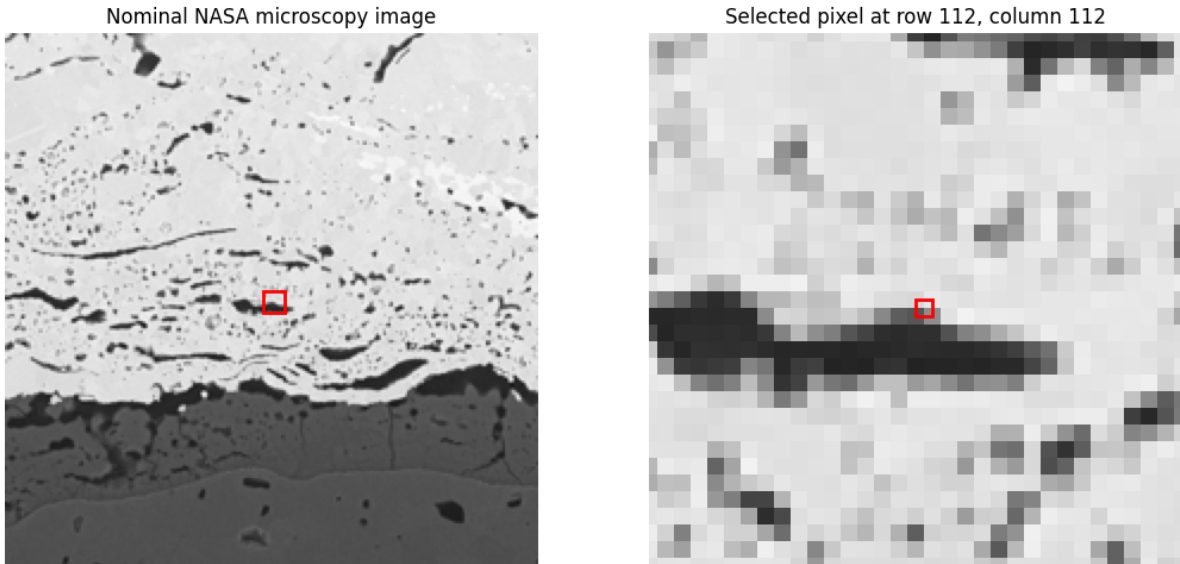


Figure 1: Nominal NASA microscopy image and the localized region originally considered for pixel-level verification. Source image: NASA MicroNet benchmark data; annotation created in the demo notebook.

3.2 Original pixel-level goal and why it changed

The first goal was to prove that an 8×8 image patch could change without changing the network’s decision. Three things got in the way. The current auto_LiRPA path could not backward-bound

¹<https://github.com/nasa/pretrained-microscopy-models>

the ResNet’s 3×3 , stride-2 MaxPool in this configuration. A sound workaround through that layer produced tight bounds, but the remaining ResNet exceeded the roughly 15 GB of GPU memory on the Colab T4. Pushing simple interval bounds deeper into the network reduced the number of variables but made the bounds far too loose: the maximum layer-3 interval width grew to about 5.1×10^5 .

None of this showed that MicroNet was unsafe. It showed that the original question was beyond this combination of verifier support, hardware, and bounding method. I therefore changed the question rather than changing the evidence: the final demo verifies the exact released linear decision layer over the 512-dimensional representation produced by MicroNet. This changes both the input being quantified over and the scope of the guarantee. In the original pixel-level formulation, the variables are raw RGB pixel values and the verifier must propagate their uncertainty through every convolution, pooling operation, residual block, and the final classifier. In the final feature-level formulation, the convolutional ResNet encoder is run once on the nominal image and is outside the formal boundary; the verifier begins with the resulting 512 learned features and proves a property only for the exact released final linear layer. The feature-level proof is therefore much smaller and tractable, but it does not imply that any particular set of pixel perturbations will remain inside that 512-dimensional feature box.

3.3 Final verified property

Formal claim.

Let z_0 represent the nominal image of Figure 1 (which was predicted Output 20, with runner up Output 30).

For every feature vector z satisfying

$$\|z - z_0\|_\infty \leq 0.083227178,$$

the exact released final layer satisfies

$$f_{20}(z) - f_{30}(z) > 0.$$

For the selected image, output 20 was highest and output 30 was the runner-up, separated by a nominal margin of 9.427921. The verification radius was set to one half of the exact L_∞ distance from the nominal feature vector to the decision hyperplane where outputs 20 and 30 tie. For a linear head, the pairwise margin has the form

$$m(z) = (w_{20} - w_{30})^\top z + (b_{20} - b_{30}),$$

so its worst-case value over $\|z - z_0\|_\infty \leq \varepsilon$ is exactly

$$m(z_0) - \varepsilon \|w_{20} - w_{30}\|_1.$$

This gives an analytic cross-check of both verifier encodings: at half the boundary distance, the worst-case margin should remain one half of the nominal margin, which is consistent with the certified value reported below. In presentation language, Marabou found no allowed 512-dimensional feature vector that can make the output-20-versus-output-30 margin less than or equal to zero.

3.4 Conclusive results

α, β -CROWN proved the condition immediately using the initial CROWN bound. The certified lower margin was 4.713961, which is strictly positive. Marabou encoded the violation, margin ≤ 0 , and returned UNSAT. The two results prove the same pairwise feature-robustness property.

Precise interpretation. All 512 learned features may vary simultaneously by up to ± 0.083227 . Output 20 is guaranteed to remain above output 30. The proof is feature-level and pairwise; it is not a pixel-level certificate and it does not establish output 20 against the other 998 competitors.

```
... Adding complete_verifier to sys.path
/content/abcrown-env/lib/python3.11/site-packages/torch/jit/_script.py:1488: DeprecationWarning: `torch.jit.script`
  warnings.warn(
INFO 22:13:37 [tensor_storage.py:70] Hugepage enabled for branch-and-bound domains.
Specification DNF: (y[0] > 0)
Model: BoundedModule(
  (0): BoundInput(name=0, inputs=[], perturbed=True)
  (1): BoundParams(name=1, inputs=[], perturbed=False)
  (2): BoundParams(name=2, inputs=[], perturbed=False)
  (3): BoundLinear(name=3, inputs=[0, /1, /2], perturbed=True)
)
Original output in the first batch: tensor([[9.42792130]])
Split layers:
Nonlinear functions:
  build batch [1/1]
Optimizable variables initialized.
initial CROWN bounds (first 10 items): [4.713960647583008]
Verified with initial CROWN!
verified with init bound!

=====
MICRONET FEATURE VERIFICATION
=====
epsilon: 0.08322717809403067
status: verified
success: True
=====

/content/abcrown-env/lib/python3.11/site-packages/maraboupy/Marabou.py:27: UserWarning: Tensorflow parser 1
  warnings.warn("Tensorflow parser is unavailable because tensorflow package is not installed")
Engine::processInputQuery: Input query (before preprocessing): 2 equations, 513 variables
unsat

=====
MICRONET FEATURE - MARABOU
=====
epsilon: 0.08322717809403067
query: margin <= 0
result: unsat
VERIFIED: no violation exists.
=====
```

α, β -CROWN output: verified with a positive lower bound.

Marabou output: violation query is UNSAT.

Figure 2: Conclusive outputs from both Colab demonstrations.

4. Case study B: NASA Airfoil Learning

4.1 Model and functional purpose

NASA’s Airfoil Learning project ² uses a neural network to estimate the aerodynamic behavior of two-dimensional airfoils. Inputs include the airfoil shape, Reynolds number, the turbulence parameter N_{crit} , and angle of attack. Outputs include lift, drag, moment, and pressure-related coefficients [3]. The repository provides code and processed datasets but not a ready-to-use checkpoint for this demonstration, so our code reproducibly retrains the published multilayer-perceptron architecture.

Here the practical question was different from classification: if three operating conditions are a little uncertain, can the predicted lift leave an acceptable tolerance band?

²<https://github.com/nasa/airfoil-learning>

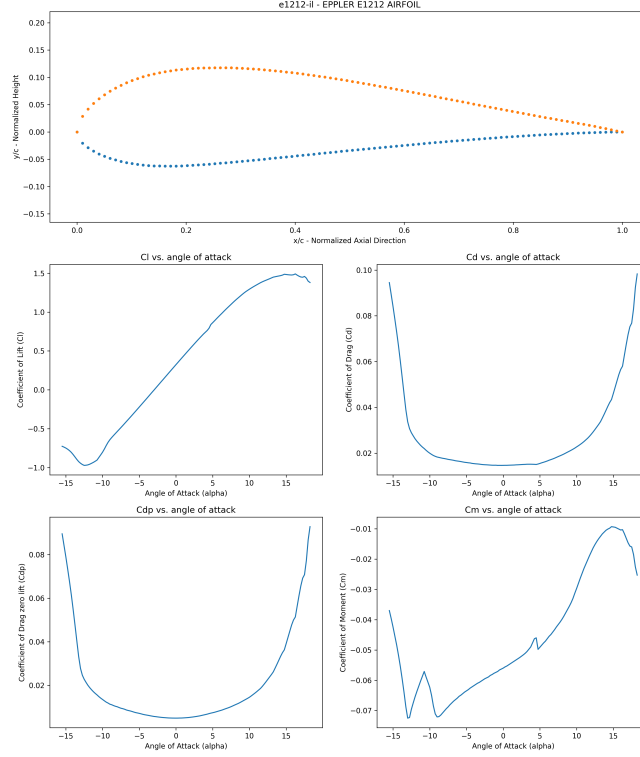


Figure 3: Example airfoil geometry and aerodynamic curves generated by the NASA Airfoil Learning workflow.

4.2 Verification boundary and property

The airfoil shape is held fixed. Its contribution to the first network layer is folded into the bias, leaving three variable operating inputs: normalized angle of attack, Reynolds number, and N_{crit} . This is an exact algebraic reduction of the trained MLP for that fixed geometry, not an approximation of its output. The nominal normalized operating point used in the final run was $(0.119318, 0.191919, 1.000000)$. Thus the verified map is a reduced 3-input network, $x \in \mathbb{R}^3 \mapsto f(x) \in \mathbb{R}^4$, with the selected output coordinate interpreted as the lift coefficient.

Formal claim. If each of the three normalized operating inputs changes by at most 0.001, the predicted lift coefficient C_l remains within ± 0.01 of its nominal value.

α, β -CROWN checks the two-sided output band directly. Marabou asks two separate questions: whether the output can fall below the lower limit and whether it can exceed the upper limit. Both Marabou queries must be UNSAT for the band to be proved.

Conclusive result. The nominal lift coefficient was 0.654780. Sound analysis placed every allowed output in $[0.654637, 0.655141]$, which lies strictly inside the required interval $[0.644780, 0.664780]$. The certified worst-case lower margin above the required lower threshold was therefore $0.654637 - 0.644780 = 0.009857$. α, β -CROWN returned **verified**. Marabou returned UNSAT for both the lower-band and upper-band violation queries. In this run, the bounded operating region also kept the relevant clipped-ReLU activation phases fixed. Over a region in which every piecewise-linear unit stays in one fixed phase, the network reduces exactly to an affine map, so its extrema over the input box can be computed soundly and directly. The reduced representation matched the trained network to within 1.2×10^{-7} on the nominal example, providing an additional numerical consistency check.

5. Case study C: official NASA text classifier

5.1 Model and functional purpose

The NASA-IMPACT `division-classifier` is a downloadable RoBERTa-based model used to tag documents for NASA’s Science Discovery Engine. It chooses among five science divisions: Astrophysics, Biological and Physical Sciences, Earth Science, Heliophysics, and Planetary Science [6].

The question for this case was: if the document’s learned 768-number representation changes slightly, does Earth Science remain the model’s top label?

5.2 Verification boundary and property

The tokenizer and large transformer encoder are run once to obtain the document’s 768-dimensional representation. The verifier then checks the exact released final classification layer. Unlike the pairwise MicroNet claim, this property compares the predicted label with every alternative label.

Formal claim. Let $z_0 \in \mathbb{R}^{768}$ be the nominal document representation. The L_∞ constraint

$$\|z - z_0\|_\infty \leq \varepsilon$$

means that every one of the 768 feature coordinates may change simultaneously, but no individual coordinate may move by more than ε in either direction. For every representation in this box, the predicted NASA science division must retain a strictly larger logit than each of the other four divisions.

The radius is chosen dynamically from the exact first decision boundary, then reduced by a factor of two for a clean positive margin. In the final run, the nominal text was: “NASA Earth-observing satellites measure changes in oceans, land, ice, and the atmosphere to improve understanding of our planet’s climate system.” The model classified it as Earth Science, with Heliophysics as runner-up, and the verification radius was 0.148291. α, β -CROWN verifies a conjunction of four strict inequalities. Marabou verifies four negated inequalities separately and requires four UNSAT results.

Conclusive result. α, β -CROWN returned **verified** with a minimum certified lower margin of 2.827524. Marabou returned UNSAT for each of the four competing divisions: Astrophysics, Biological and Physical Sciences, Heliophysics, and Planetary Science. Thus Earth Science remains the top output throughout the complete 768-feature box. Equivalently, Marabou found no allowed 768-dimensional feature vector that can make any Earth-Science-versus-competitor margin less than or equal to zero. This remains a feature-level proof; it is not a guarantee over arbitrary word or token substitutions.

6. Case study D: NASA GIBS anomaly detection

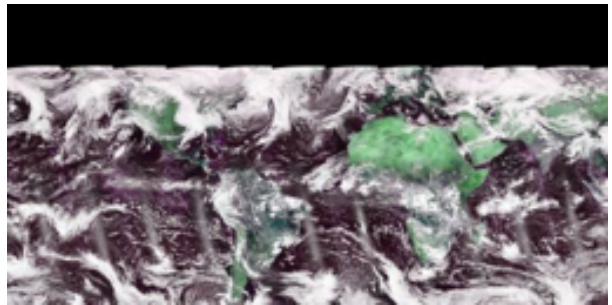
6.1 Model and functional purpose

NASA’s Global Imagery Browse Services (GIBS) [4, 5] gives users large-scale, near-real-time access to Earth-observation imagery. Its machine-learning project explores data-validation problems such as missing regions and incorrect coloring, along with natural-event detection [4, 5]. One published approach combines hand-designed texture and color features with a two-layer fully connected neural classifier [4]. In the repository’s handcrafted-feature workflow, each satellite image is converted into numerical descriptors before classification: a Histogram of Oriented Gradients (HOG) summarizes local texture while a hue histogram in HSV color space summarizes color. The repository then evaluates several classifiers on these features, including a two-layer fully connected neural network with a Softmax output. This is therefore a feed-forward multilayer perceptron operating on engineered image features, not an end-to-end convolutional network operating directly on raw pixels. In the reproducible demonstration below, the formalized classifier uses the 256-bin hue histogram as its input and a $256 \rightarrow 100 \rightarrow 2$ ReLU network to produce normal-versus-anomaly scores.

The practical question was: if every color-histogram feature changes within a small allowed range, can a nominal image cross the classifier’s anomaly boundary?



Nominal daily image.



Controlled hue-miscoloration example.

Figure 4: Real NASA GIBS imagery used in our code and its controlled anomaly transformation.

6.2 Reproducible anomaly demo

The GIBS repository publishes the architecture and workflow, but not a downloadable checkpoint suitable for this comparison. Our code therefore downloads real daily VIIRS imagery, creates controlled hue-miscoloration examples, calculates a 256-bin hue histogram, and retrain a $256 \rightarrow 100 \rightarrow 2$ ReLU classifier following the published design. The histogram calculation sits outside the formal boundary; the complete trained neural classifier is inside it.

Formal claim. For every 256-feature hue histogram within the selected L_∞ box around a nominal image, the normal score remains greater than the anomaly score.

A final reproducibility run selected 15 December 2022 as the nominal example and produced a nominal normal-minus-anomaly margin of 5.384054. The notebook analytically selected a conservative radius, $\varepsilon = 0.002594$, that kept every hidden ReLU state stable. The exact affine lower-margin cross-check was 3.830791. The controlled train, validation, and test splits each achieved 100% accuracy; those accuracy values describe this constructed anomaly task and should not be interpreted as performance on unconstrained operational anomalies.

Conclusive result. α, β -CROWN returned **verified**, and Marabou returned **UNSAT** for the violation query. Therefore every feature vector in the selected 256-dimensional box keeps the normal score above the anomaly score. Equivalently, Marabou found no allowed hue-histogram feature vector that can make the normal-versus-anomaly margin less than or equal to zero. This is a complete-network proof for the trained ReLU classifier; the hue-histogram feature extraction remains outside the formal boundary.

7. What I learned from the experiments

- **The question matters as much as the model.** A smaller, clearly defined claim can produce a useful guarantee; an ambitious but unresolved query cannot.
- **The verification boundary must be visible.** Feature-level, pairwise, all-competitor, and complete-classifier proofs answer different questions.
- **An inconclusive run is not a counterexample.** The full MicroNet query ran into software and memory limits. That says something about the experiment, not automatically about the network’s robustness.
- **Model conversion needs a check.** Before using ONNX in Marabou, the notebook compares its nominal output with PyTorch. The MicroNet difference was about 7.6×10^{-6} .
- **Hard-coded labels are fragile.** When repository image ordering changed, the MicroNet runner-up changed too. The final notebook discovers the nominal prediction and competitor at runtime.
- **Two tools help catch setup mistakes.** A positive certified bound and an UNSAT violation query should describe the same mathematical property.

8. Code and reproducibility

The complete notebook and supporting material are publicly available in the project repository [11]:

`https://github.com/saralogsdon/neural-network-verification`

The notebook creates a Python 3.11 environment for α, β -CROWN and Marabou, downloads or retrains the required models, runs both verifiers, and packages the JSON evidence, CSV summary, ONNX models, metadata, and images. The repository also includes this paper and the presentation so that the claims, code, and audience-facing summary can be checked together.

Reproducibility here means more than making the notebook public. The notebook records the model source, dynamically discovers labels instead of assuming fixed indices, compares PyTorch and ONNX outputs, prints the exact epsilon and margin, and distinguishes official checkpoints from retrained NASA architectures. A future run may produce slightly different trained weights or labels, but it should still produce evidence with the same structure and interpretation.

9. Next steps

The next useful step is not just to add more demos. It is to make the verified properties more system-relevant and to understand exactly when the current tools scale.

- **Push MicroNet back toward pixels.** Revisit the original 8×8 patch query with stronger hardware, improved MaxPool support, alternative encodings, or a decomposed proof that verifies only part of the ResNet at a time. Track whether the limiting factor is operator support, memory, or bound looseness.
- **Run systematic epsilon sweeps.** For each case, record the largest radius that can be verified, the runtime, the number of subproblems or splits, and whether the proof used only initial bounds, optimized α bounds, or complete branch-and-bound with β splits.
- **Replace arbitrary robustness radii with domain requirements.** For airfoil, this means tolerances tied to engineering use; for GIBS, realistic histogram variation or known image-quality shifts; for text, representation changes motivated by actual document variation.
- **Make pairwise, all-competitor, regression-band, and complete-network claims comparable.** The current four examples are useful precisely because they illustrate different specification types; the next step is to document which types are most meaningful for NASA assurance cases.
- **Connect model-level certificates to system-level assurance.** A verified neural component could become an assumption inside a larger PVS, NASALib, Plaidypvs, or runtime-assurance model. This is the bridge between a local proof and an autonomy V&V argument.
- **Standardize evidence packaging.** Each run should save the model provenance, exact verification boundary, input bounds, specification, solver settings, runtime, result, and counterexample or certificate information in a format that another engineer can audit.

10. Conclusions

- **MicroNet:** We conclusively verified a pairwise robustness property of the exact released decision head over all 512 learned features, using two different formal approaches.
- **Airfoil:** Both tools proved that bounded uncertainty in three operating variables cannot push the exact reduced NASA-architecture MLP’s lift prediction outside the chosen tolerance band.
- **Text:** Both tools proved that the official NASA classifier’s exact final layer preserves the Earth Science decision against all four competing divisions within the selected representation region.
- **GIBS:** Both tools proved that the complete retrained NASA-style anomaly classifier preserves the normal decision over bounded uncertainty in all 256 hue-histogram features.

Evidence-package checklist

1. The evidence manifest reports all six verifier records available and no missing assets.
2. Every α, β -CROWN record reports **verified**; every required Marabou violation query reports **UNSAT**.
3. The package retains JSON evidence, the CSV summary, ONNX models, metadata, representative images, and saved nominal feature vectors.
4. Report the exact epsilon, margin, and model boundary from the evidence rather than generalizing beyond the proved claim.
5. Keep the provenance language: feature-level versus complete-network, pairwise versus all-competitor, and official checkpoint versus retrained architecture.

11. Conclusion: what this project actually shows

This project produced four conclusive demonstrations in which α, β -CROWN and Marabou independently verified the same stated property. The properties covered pairwise feature robustness, a two-sided regression tolerance, all-competitor text classification, and a complete small anomaly classifier. Those are real guarantees over infinitely many inputs inside the selected regions, not results from sampling alone.

The project does not show that every NASA neural network can already be verified from raw input to final decision. The full MicroNet attempt is the clearest reminder: model size, unsupported operations, loose bounds, and available memory all shape what is practical. The response was not to hide the failed attempt or overstate the smaller proof. It was to identify a tractable boundary and describe it honestly.

Formal verification is not a button that declares a model safe. It is a disciplined way to ask one precise question, check every input inside a defined region, and receive a conclusion whose usefulness depends on how well the question matches the real system.

References

- [1] NASA, *pretrained_microscopy_models repository*.
<https://github.com/nasa/pretrained-microscopy-models>
- [2] Stuckner, Harder, and Smith, “Microstructure segmentation with deep learning encoders pre-trained on a large microscopy dataset,” *npj Computational Materials*, 2022.
<https://www.nature.com/articles/s41524-022-00878-5>
- [3] NASA, *Airfoil Learning repository*.
<https://github.com/nasa/airfoil-learning>
- [4] NASA GIBS, *gibs-ml repository*.
<https://github.com/nasa-gibs/gibs-ml>
- [5] NASA Software Catalog, *GIBS Machine Learning for Data Validation and Natural Event Detection*, NPO-51089-1.
<https://software.nasa.gov/software/NPO-51089-1>
- [6] NASA-IMPACT, *Division Classification Model card*.
<https://huggingface.co/nasa-impact/division-classifier>
- [7] Verified Intelligence, *α, β -CROWN repository and high-level API documentation*.
<https://github.com/Verified-Intelligence/alpha-beta-CROWN>
- [8] Wang et al., “Beta-CROWN: Efficient Bound Propagation with Per-neuron Split Constraints for Complete and Incomplete Neural Network Robustness Verification,” 2021.
<https://arxiv.org/abs/2103.06624>
- [9] Neural Network Verification, *Marabou repository and Python documentation*.
<https://github.com/NeuralNetworkVerification/Marabou>
- [10] Wu et al., “Marabou 2.0: A Versatile Formal Analyzer of Neural Networks,” 2024.
<https://arxiv.org/abs/2401.14461>
- [11] Logsdon, Sara, *Neural Network Verification: NASA-related demonstrations with α, β -CROWN and Marabou*, 2026.
<https://github.com/saralogsdon/neural-network-verification>